



High Performance Computing Exercise Sheet 12

HS 25
Dr. Douglas Potter

<http://www.ics.uzh.ch/>

Teaching Assistants:

Cesare Cozza, cesare.cozza@uzh.ch

Mark Eberlein, mark.eberlein@uzh.ch

Issued: 05.12.2025

Due: 12.12.2025

Exercise 1 [Science Cloud]

In this exercise, you will create and set up a virtual machine (VM) instance on the Science Cloud.

- a) Connect and login to <https://cloud.science-it.uzh.ch> with your UZH Shortname and your Webpass password. In case you have no access to the cloud please write us a mail with your uni-mail and shortname.

Create an Ubuntu 24.04 instance with 2 cores and 8 GB of RAM (hpcv3 flavour). You can follow, step by step, the instructions given in the lecture slides.

Once the instance has been created, you can login through `ssh` with the IP address relative to your instance. Remember that you need to connect to the UZH VPN first if your outside of the university network. If you haven't set up your VPN yet, follow these instructions: https://www.zi.uzh.ch/de/support/netzwerk/vpn_ISAC.html

Finally, update the list of packages and install the `cowsay` utility and test that your Ubuntu installation works fine.

Commit : Upload a screenshot showing what the cow says on your cloud instance.

- b) Now we are going to compile and run the serial Poisson solver code on the VM. Clone the course git repository, https://github.com/Mark-Eb/hpc_esc_401_2025 in your VM, and move to the `exercise_session_12/poisson_solver_serial` folder. In order to compile the code however, we need:

- A compiler: install the `g++` compiler in your VM.
- Since we use a Makefile to compile the code, we need the `make` tool: install it too

Then you should be able to compile the code and then run it.

- c) To visualize your output files, you can use the Python script `plot.py` (in the `output` folder). Before we can run the script, we need to install the required packages (`numpy`, `matplotlib`). In the following, we go through the installation process using the `venv` environment manager . If you are more familiar with `conda` you can use it instead.

- (i) Check with `python3 --version` that python is correctly installed.
- (ii) Install `python3-pip` and `python3-venv` via `apt`.
- (iii) Create a new virtual python environment using the command `python3 -m venv <PATH/YOUR_ENV_NAME>`, where `<PATH/YOUR_ENV_NAME>` is the path where you want to store your environment.
- (iv) Activate your environment using `source <PATH/YOUR_ENV_NAME>/bin/activate`.
- (v) Install the required python packages using `pip`. If installation of any python package fails with error `Failed building wheel...`, consider updating wheel using `python3 -m pip install --upgrade pip setuptools wheel`.

Commit : Run the serial Poisson solver and upload the `test.png` image produced by the `plot.py` script.

- c) Create and save a snapshot of your instance on the Science Cloud. Name the snapshot something like `YourName_PoissonOnUbuntu` so we can check that it exists.

BONUS: Try now to compile and run programs parallelized with MPI or OpenMP. You can use either your own versions of the Poisson solver or the solutions provided in the course repository.

Exercise 2 [Containers]

- a) Install Docker on your laptop and on your instance on the science cloud following these instructions: <https://docs.docker.com/get-docker/>.
- b) **Commit :** (from this point below) To build a Docker image of the serial Poisson solver on your laptop, you have to define a container with a Dockerfile. You can think of Dockerfile commands as a step-by-step recipe on how to build up your image (see the lecture slides for a simple example). Here are the main Dockerfile commands that you should use:
 - FROM: This command must be on top of the Dockerfile and specifies the base image that you will use to build your image. In our case you can use the `ubuntu:trusty` base image.
 - RUN: Can be used to run commands, such as installing the dependencies of your code (`make`, `g++...`), or compiling it, during the build process.
 - COPY: Allows you to copy your source code in the image.
 - WORKDIR: Specifies the directory from where you want to run any following command.
 - CMD: Contains the run-time commands of your container, basically launching the code in your case.

Create a Dockerfile (named `Dockerfile`) with the necessary instructions inside the root directory of the Poisson solver directory and execute the command "`docker build --tag poisson .`" to build your docker image. Make sure that your image is correctly build with "`docker images`".

Run the container on your local machine with `docker run` to test your setup. If you get an error about a missing `libstdc++` library try to specify the Ubuntu 24.04 version in your Dockerfile. If everything works you may notice that the Output directory in your computer does not contain any new file. This is because files were written in the container. In order to access the output files, you need to mount a host directory into the container. To do that, use the `-v` or `--mount` flags and specify the host directory you want to link to the container one.

- c) Now that your Docker image is setup, you can push it to your Docker Hub repository. First setup a Docker Hub account, then create a new repository named `poisson` on the web page. To push the docker image, you first have to rename your image like `<Docker ID>/<Repository Name>:<tag>`, to do so you can run the command `docker tag poisson:latest <Docker ID>/poisson:latest`. Finally push the image with `docker push <Docker ID>/poisson:latest`
- d) Pull the Docker image from the Docker Hub registry and run it on your VM. Install instructions <https://docs.docker.com/engine/install/ubuntu/>, post install instructions (Manage Docker as a non-root user) <https://docs.docker.com/engine/install/linux-postinstall/>

Exercise 3 [*Bonus*: Delete your instance]

Once your done with the sheet, delet your instance on the ScienceCloud.